

API calling using useEffect

1. To create an useEffect hook
2. define the dependencies
3. call the API
4. update the state which needs to hold the API response (optional)

we can not directly create the useEffect function as async directly because it has to be a normal function as per the React definition of useEffect

```
useEffect(()=>{  
  // create a function which is async  
  // call the function immediately after declaration  
}, [])
```

In order to show the data on the webpage, we use a state to store and use the data at a later stage when it is available.

Where to fetch the data?

- useEffect

Why useEffect to fetch the data?

- it is called after component mounts

What is a key?

A key is a unique identifier for each item in the list.

It is required to be passed especially when rendering the list using the .map().

Keys help React to identify which items have changed, added or removed.

Without keys React cannot track elements properly and throws a warning that Each child in the list should have a key prop.

Rules for a key:

1. Keys must be unique among siblings
 - user.id
2. Keys must be stable - they should not change in between renders.
 - database uuid
 - uuid stored in the data
 - unique combination for each item
3. Avoid using the array indexes as key

Problem:

- breaks when items are reordered
- react. can reuse wrong DOM elements

react can render many DOM elements

Allowed in specific cases:

- list is static
- no reordering / addition / removal of elements

React Forms

React forms are controlling the input fields and managing the state, validating data and handling the submission of forms.

instead of using individual states for each field we use an object containing key value pair for all the fields where key is referred by the name property of the input field

Best Practices for React Forms

1. Use one state object to manage multiple field data
 - cleaner and scalable
2. Use 'name' attribute as object keys
 - one change handler can be used to update all the fields
3. Prevent Default on submit
 - it avoids page reload
4. Validate before submitting
 - ensures data is secured and required fields have the data.
5. Use controlled components
 - using input values coming in from React state

useCallback

React is all about rendering and re-rendering.

Memoization - it is an optimization technique which is used to speed up the program by storing the result of expensive function calls and resulting the cached result when the same inputs are received.

How it works?

1. when a function call is made, memoization checks whether the result for those inputs is already saved
2. if found, returns the result from cached data
3. if not, computes the result and then returns the data also saving it to cache

useCallback is used to memoize the functions meaning the function is not created on every render unless certain dependencies change.

1. on every re-render in React, the functions are redefined and this becomes inefficient if the function is passed as a prop to child component as it will trigger unnecessary updates without any dependency on the function.
2. useCallback will avoid unnecessary creation of function by memoizing them it returns memoized version that changes when one of the dependency is changed

```
const memoizedFunction = useCallback(() => {  
  // ...  
}, [dependencies]);
```

```
// logic here  
}, [dependencies])
```

Recreation prevention - useCallback memoizes the function so it does not get recreated

Optimization - It can help avoid unnecessary renders of child component or unnecessary re-evaluation of side effects that depend on the function

Dependencies - ensure the dependencies are marked appropriately because if unnecessary dependencies are included it can lead to missed updates.

useMemo

useMemo is React hook used to memoize result of a computation so that the computation is only re-run when any of the dependency is changed.

It is useful for expensive calculations or when we have to maintain stable values between renders

How useMemo works?

1. runs a function only when the dependencies are changed
2. returns the memoized value if same inputs received
3. prevents unnecessary recalculations on every render